

Lecture 16: Prototypes, Classes, and Event Loops

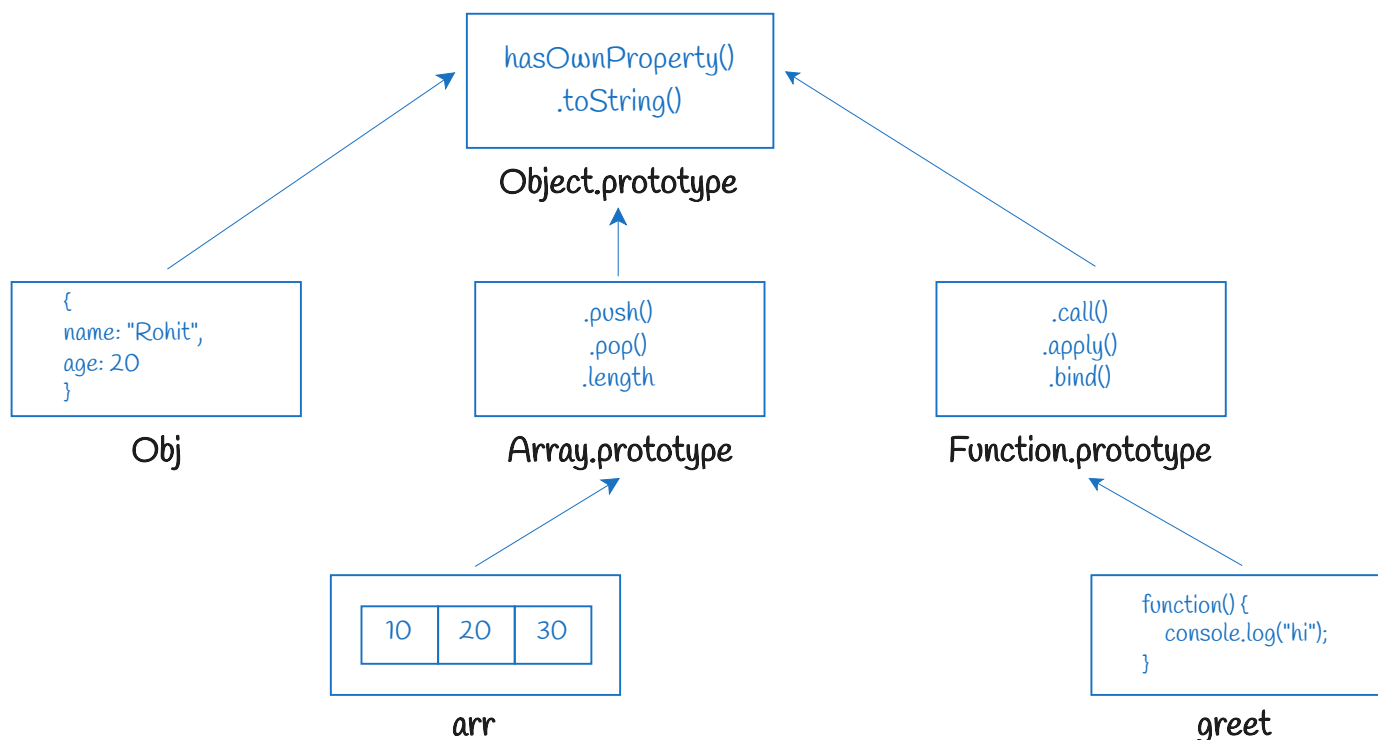
Prototypes and the Prototype Chain:

When an object or array is created in JavaScript, it automatically gains access to built-in methods (like `.hasOwnProperty()`, `.toString()`, `.push()`, or `.pop()`) even though they were never explicitly defined by the developer.

- **The Mechanism:** This happens because of a hidden, pre-built property called `[[Prototype]]`.
- When a new object is created, JavaScript attaches `Object.prototype` to it behind the scenes. This prototype is simply another object that contains a library of common methods.
- If a developer calls a method that doesn't exist on the immediate object, the JavaScript engine looks "up" the chain into the prototype to find it.
- **The Benefit:** By sharing a single prototype object across all instances (instead of creating a new copy of every method for every array or object), JavaScript saves massive amounts of memory.

The Prototype Hierarchy (Chain):

- An Array inherits from `Array.prototype`.
- `Array.prototype` inherits from `Object.prototype`.
- `Function.prototype` inherits from `Object.prototype`.
- `Object.prototype` inherits from null (this is the end of the chain).



Note: Custom prototypes can also be set manually using `__proto__`.

Classes and Object-Oriented Programming (OOP):

Manually writing out objects with the exact same keys repeatedly is inefficient. A **Class** solves this by acting as a blueprint for creating multiple objects with the same structure.

1. Constructors and this:

- The constructor is a special method inside a class used to initialize values when a new object is created.
- The `this` keyword refers to the empty object currently being created by the `new` keyword, allowing properties to be attached directly to it (e.g., `this.name = name`).

2. Methods in Classes:

- When a function (method) is defined inside a class, it is not copied into every single object instance.
- Instead, it is stored in the Class's prototype. All objects created from that blueprint share access to that one method, optimizing memory usage.

3. Inheritance (extends and super):

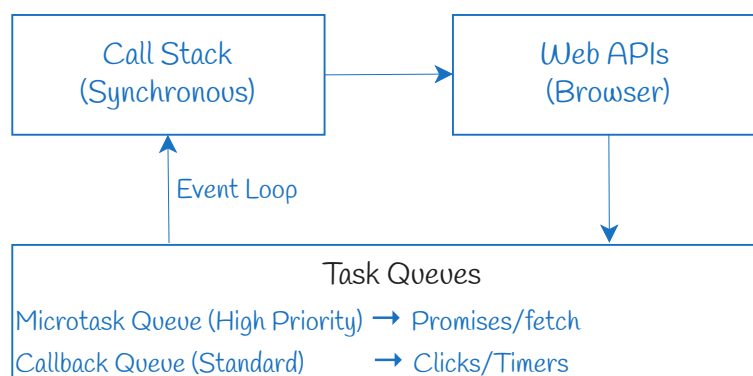
- A child class can inherit properties from a parent class using the `extends` keyword.
- To initialize the parent's properties, the `super()` method must be called inside the child's constructor.

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
};  
  
class Customer extends Person {  
  constructor(name, age, balance, city) {  
    super(name, age);  
    this.balance = balance;  
    this.city = city;  
  }  
};
```

The Event Loop and Asynchronous Execution:

JavaScript is a single-threaded language, meaning its **Call Stack** can only execute one task at a time. To handle asynchronous tasks (like listening for three different button clicks simultaneously) without freezing, JavaScript relies on the Browser's multithreaded architecture.

The Architecture Breakdown:



1. **Call Stack:** Executes standard, synchronous JavaScript code line-by-line.
2. **Web APIs (Browser):** When the Call Stack encounters an asynchronous task, it offloads it to the Browser's Web APIs. The Browser handles the waiting/listening in the background, freeing up the Call Stack.
3. **Callback Queue:** Once a standard Web API task finishes (e.g., a button is clicked), its callback function is moved into this queue to wait for execution.
4. **Microtask Queue:** A specialized queue with higher priority than the standard Callback Queue. It handles specific operations like Promises (fetch data).
5. **The Event Loop:** A continuous background process that monitors both the Call Stack and the Queues. Crucially, it will only move a function from a queue into the Call Stack if the Call Stack is completely empty.

Why wait for an empty Call Stack?

If asynchronous callbacks were injected directly into the Call Stack while standard code was still running, it would interrupt previous instructions and cause unpredictable, non-deterministic behaviour. The Event Loop ensures smooth, orderly execution.